Lab 2 AC

Abel Saiz Eduardo Salado

Descripción de la sesión **3**
Cálculo de FLOPs **3**
Pruebas **4**
Prueba 1 (base) 4
Prueba 2 (4 hilos) 4
dynamic 4
guided 5
static 5
Prueba 3 (8 hilos) 6
dynamic 6
guided 6
static 6
prueba 4 (12 hilos) 7
dynamic 7
guided 7
static 8
prueba 4 (24 hilos) 9
dynamic 9
guided 9
static 9

Descripción de la sesión

En esta sesión abordaremos la paralelización del código secuencial desarrollado previamente utilizando el estándar OpenMP. Para lograrlo, incluiremos la librería correspondiente y añadiremos las directivas pragma necesarias en los bucles clave de nuestro algoritmo. El estudio experimental consistirá en evaluar el rendimiento variando tanto el tamaño del problema como el número de hilos. Además, analizaremos distintas políticas de planificación (*schedule*) para determinar empíricamente cuál es la más eficiente. Aunque el análisis teórico previo nos permite deducir qué configuración será la óptima, resulta fundamental realizar estas pruebas para extraer datos precisos, generar gráficas representativas y justificar el comportamiento real del sistema.

Cálculo de FLOPs

La paralelización del algoritmo mediante OpenMP no altera la cantidad total de operaciones matemáticas a realizar, sino la forma en que estas se distribuyen entre las distintas unidades de procesamiento. Por consiguiente, el coste computacional total (FLOPs), se mantiene idéntico al de la implementación secuencial desarrollada en la Práctica 0.

$$\text{FLOPs} = 6 \cdot N \cdot D^2 + N^2 \cdot (4D + 11) + 4N + 12$$

Para valores grandes de N y D, los términos dominantes de la ecuación son $N \cdot D2$ y $N2 \cdot D$. Por lo tanto, el orden de complejidad asintótica del algoritmo se define como $O(N \cdot D2+N2 \cdot D)$.

Pruebas

Para las pruebas dejaremos para el final la que creemos que es el mejor resultado de forma teórica y así lo comprobaremos. También añadiremos al final, una opción más cambiando el código original. Haremos distintas pruebas variando hilos y N para cada planificación

Prueba 1 (base)

Aquí analizaremos el caso base sin openMP, es el mismo que el óptimo de la práctica anterior pero lo volveremos a poner.

Talla (N, D=64)	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,057908	1364,933308
1000	291.580.012	0,239296	1218,490957
2000	1.117.160.012	0,93037	1200,769599
4000	4.370.320.012	3,68162	1187,064393
8000	17.284.640.012	14,9942	1152,755066

Analizamos que sin ningún tipo de paralelización, los tiempos son los mismos a la práctica anterior. Estos datos los usaremos únicamente para comparar con las otras pruebas.

Prueba 2 (4 hilos)

Comenzaremos el análisis del código paralelizado con 4 hilos, probando cada tipo de planificación. Hemos incorporado el tamaño de problema de 20.000, ya que los tamaños anteriores no permitían justificar adecuadamente por qué consideramos que la planificación static es la más apropiada.

dynamic

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,016844	4692,389236
1000	291.580.012	0,068723	4242,817738
2000	1.117.160.012	0,25367	4404,07629
4000	4.370.320.012	0,96073	4548,938637
8000	17.284.640.012	3,9511	4374,684265
20000	107.291.600.012	24,7335	4337,906079

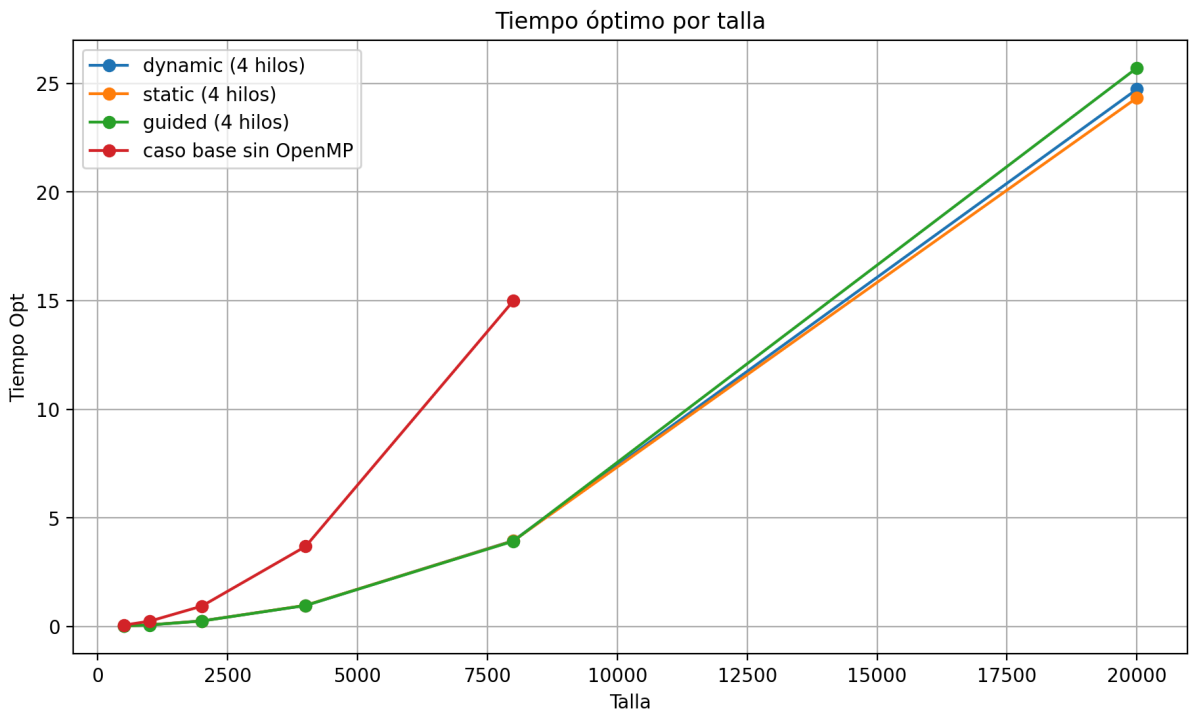
guided

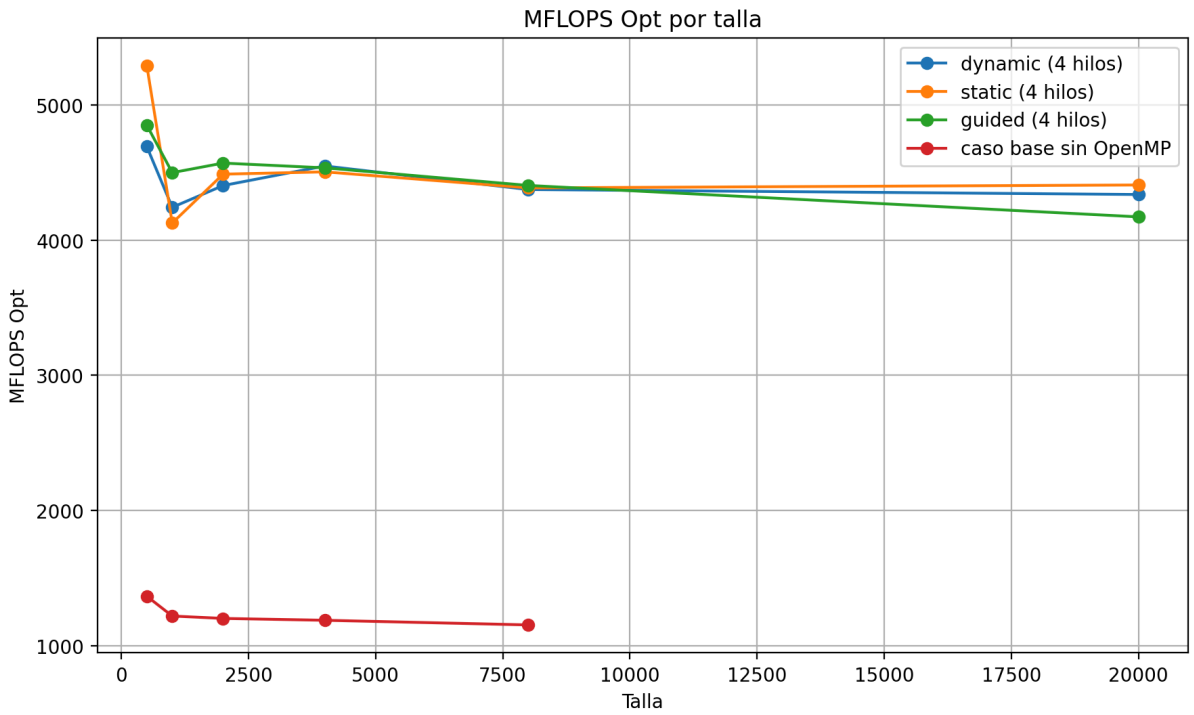
talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,016296	4850,300505
1000	291.580.012	0,064801	4499,642936
2000	1.117.160.012	0,24444	4570,339237
4000	4.370.320.012	0,96366	4535,112391
8000	17.284.640.012	3,9239	4404,953226
20000	107.291.600.012	25,7146	4172,400116

static

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,014939	5290,991927
1000	291.580.012	0,070656	4126,73763
2000	1.117.160.012	0,24891	4488,298797
4000	4.370.320.012	0,97000	4505,465969
8000	17.284.640.012	3,9402	4386,752926
20000	107.291.600.012	24,3382	4408,362164

Evidentemente la mejoría frente a el caso base es notoria, viendo que ronda los 4380 MFLOPs más o menos, que es ya 4 veces mejor que la base, lo cual tiene sentido porque son 4 hebras. La mejora con respecto al caso base es claramente significativa, observándose un rendimiento de aproximadamente 4380 MFLOPs. Esto representa un aumento de rendimiento de unas 4 veces el caso base, lo cual es coherente al haberse utilizado 4 hilos.Es cierto que los tiempos observados son casi idénticos y no presentan una diferencia notable. Esto se debe a la baja cantidad de hilos y al tamaño reducido de las tareas. Por esta razón, el sobrecoste real de utilizar la planificación dynamic frente a static cuando no es necesario aún no es evidente.





Prueba 3 (8 hilos)

ahora realizaremos las mismas pruebas pero probando con 8 hilos.

dynamic

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.

500	79.040.012	0,009562	8265,630814
1000	291.580.012	0,036425	8004,920014
2000	1.117.160.012	0,13174	8479,779056
4000	4.370.320.012	0,50924	8582,027001
8000	17.284.640.012	2,0530	8419,087891
20000	107.291.600.012	12,9717	8271,20578

guided

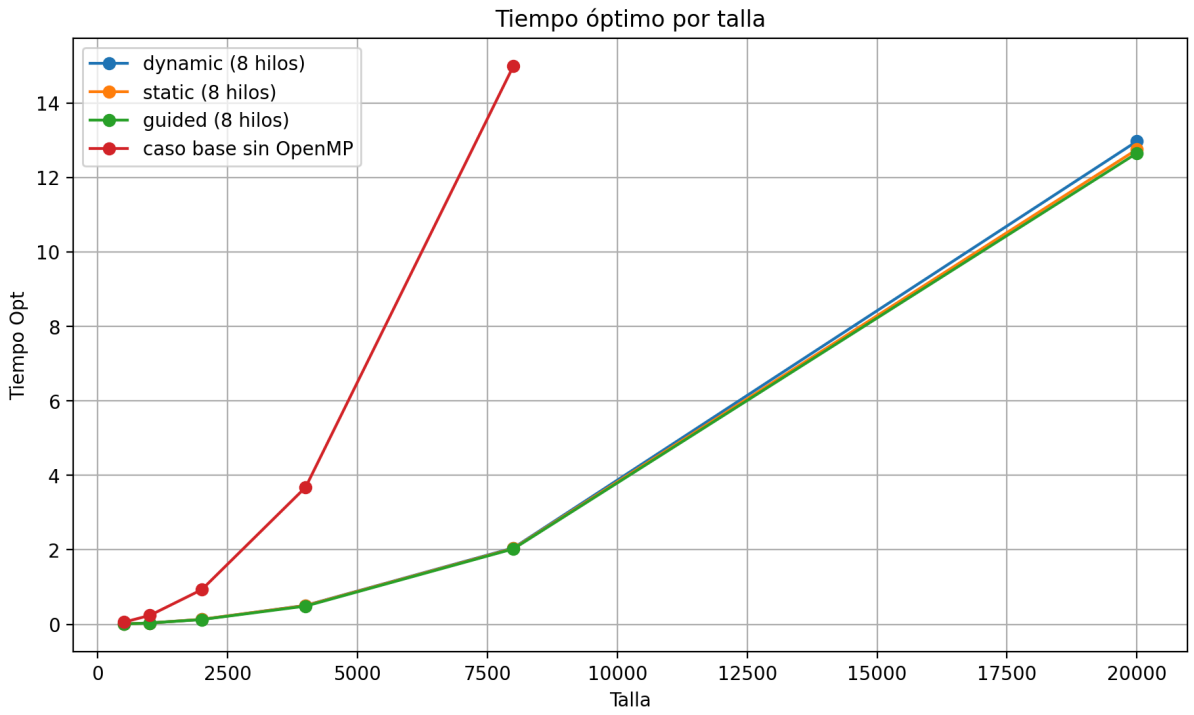
talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,007909	9994,210339
1000	291.580.012	0,036036	8091,465694
2000	1.117.160.012	0,12851	8693,446314
4000	4.370.320.012	0,49971	8745,677534
8000	17.284.640.012	2,0253	8534,233933
20000	107.291.600.012	12,6525	8479,873544

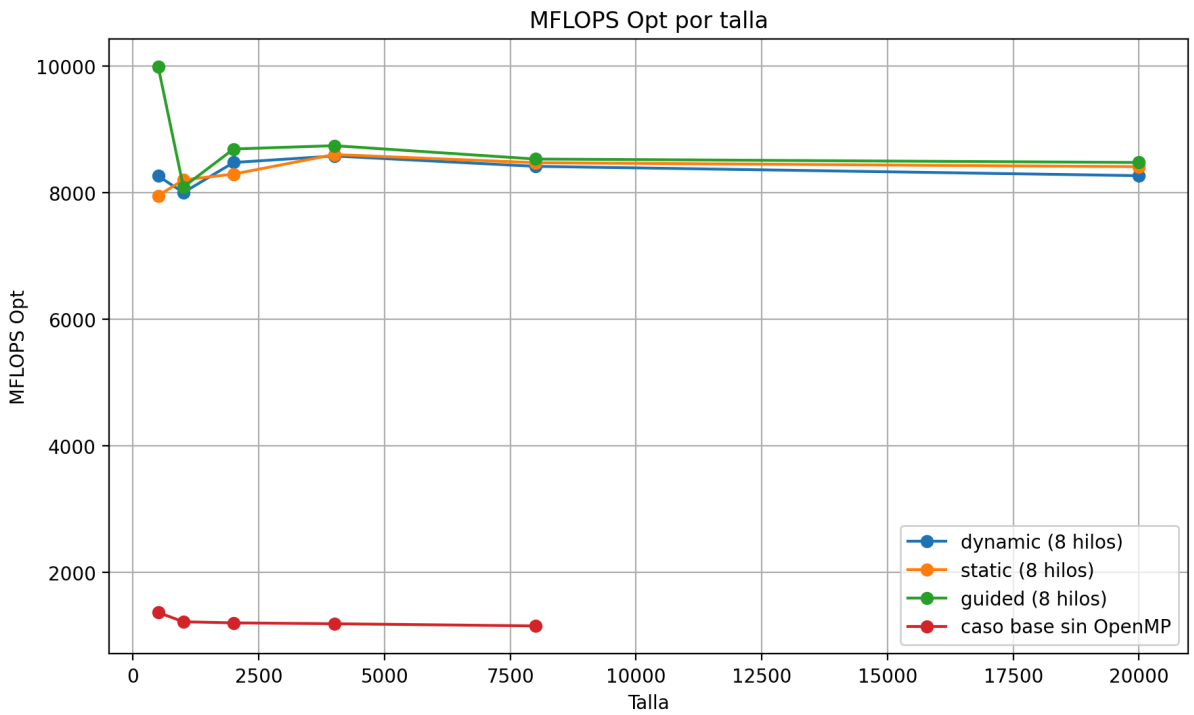
static

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,009936	7955,216886
1000	291.580.012	0,035535	8205,454694
2000	1.117.160.012	0,13465	8296,646259
4000	4.370.320.012	0,50437	8664,891542
8000	17.284.640.012	2,0391	8476,519289
20000	107.291.600.012	12,7526	8413,311796

Continuando con la tendencia positiva, al emplear 8 hilos observamos que el rendimiento prácticamente se duplica respecto al caso anterior, alcanzando aproximadamente los 8470 MFLOPS. Esto supone un incremento de rendimiento cercano a 8 veces el caso base secuencial. Esta escalabilidad casi lineal es posible gracias a las especificaciones del servidor: al contar con 12 núcleos físicos reales, los 8 hilos siguen ejecutándose de forma totalmente paralela y disponen de un núcleo dedicado en exclusiva para cada uno, sin penalizaciones por compartir la memoria caché.

En cuanto a las distintas políticas de planificación (static, dynamic, guided), los tiempos de ejecución siguen manteniéndose muy parejos. Aunque el número de hebras concurrentes ha aumentado, todavía no es una cantidad suficiente para generar un cuello de botella crítico en la asignación dinámica de tareas. El enorme peso computacional de cada iteración sigue enmascarando el ligero sobrecoste (overhead) que introduce la planificación dynamic, haciendo que en este escenario aún no se aprecie una degradación del rendimiento frente a static.





prueba 4 (12 hilos)

dynamic

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,006658	11871,71996
1000	291.580.012	0,025759	11319,53927
2000	1.117.160.012	0,09861	11328,86746
4000	4.370.320.012	0,35559	12290,4381
8000	17.284.640.012	1,3804	12521,10922
20000	107.291.600.012	9,3310	11498,44014

guided

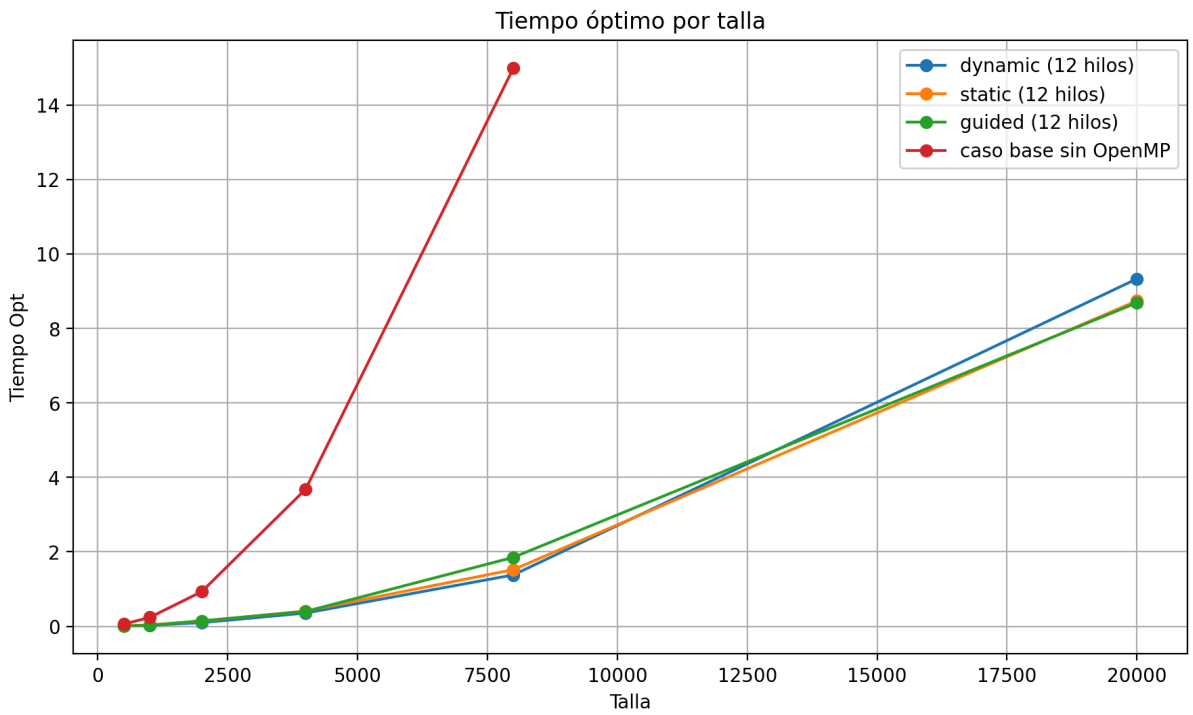
talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,008140	9709,967826
1000	291.580.012	0,029727	9808,7569
2000	1.117.160.012	0,14633	7634,785662
4000	4.370.320.012	0,39848	10967,44892
8000	17.284.640.012	1,8522	9332,001583

20000	107.291.600.012	8,6878	12349,7142

static

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,008978	8803,851685
1000	291.580.012	0,036168	8061,756068
2000	1.117.160.012	0,13670	8172,348296
4000	4.370.320.012	0,41104	10632,42485
8000	17.284.640.012	1,5225	11352,72676
20000	107.291.600.012	8,7403	12275,54755

Al analizar los resultados para 12 hebras , observamos un comportamiento atípico: la planificación static presenta un rendimiento inferior al esperado (incluso rindiendo peor que con 8 hilos en tallas de 1000 y 2000), mientras que dynamic logra superarla. Esto evidencia la naturaleza NUMA de los dos procesadores del servidor. Al inicializarse las matrices secuencialmente (First Touch), los datos residen en el nodo del primer procesador. Con static, se fuerza al segundo procesador a procesar el 50% de la carga accediendo a memoria remota, lo que introduce una enorme latencia y frena la ejecución. Por el contrario, dynamic logra mitigar esta penalización, permitiendo que los hilos del nodo local (más rápidos) procesen un mayor porcentaje de la carga, demostrando su utilidad como balanceador de carga frente a latencias de memoria.



prueba 4 (24 hilos)

dynamic

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,005628	14044,99088
1000	291.580.012	0,023268	12531,26636
2000	1.117.160.012	0,08482	13171,19884
4000	4.370.320.012	0,32561	13421,90532
8000	17.284.640.012	1,2673	13639,37946
20000	107.291.600.012	7,3382	14620,89002

guided

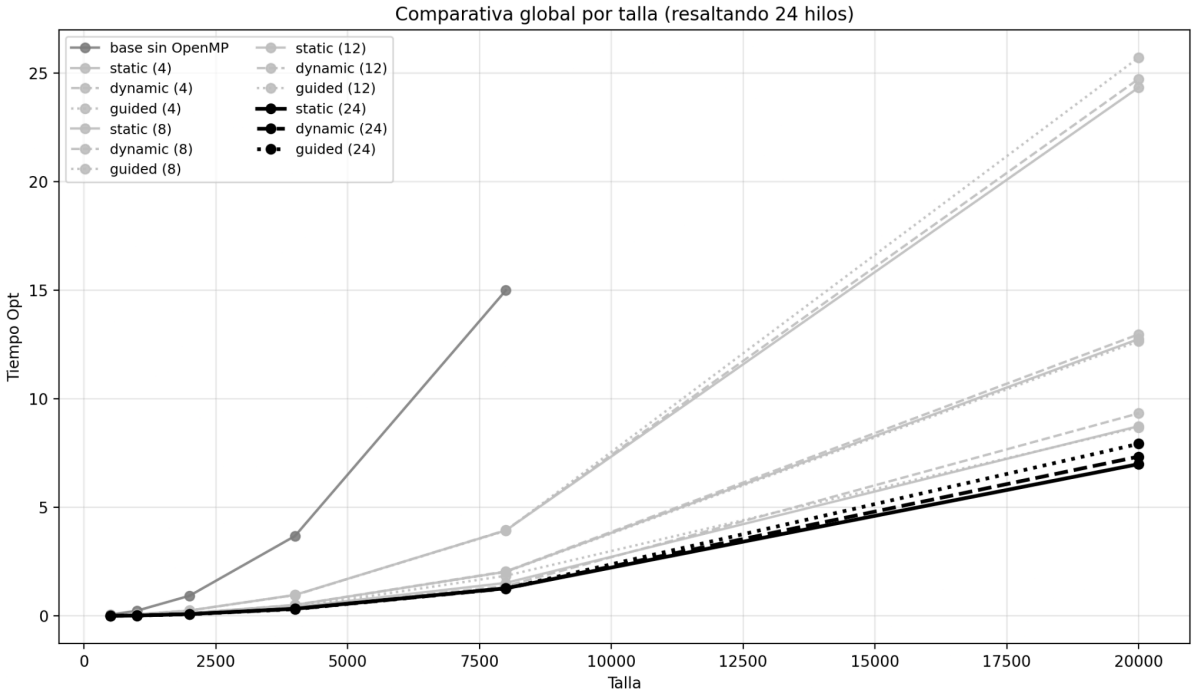
talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,006177	12796,45831
1000	291.580.012	0,019407	15024,63116
2000	1.117.160.012	0,08066	13850,7852
4000	4.370.320.012	0,31559	13848,09408
8000	17.284.640.012	1,2695	13615,20588
20000	107.291.600.012	7,9273	13534,46134

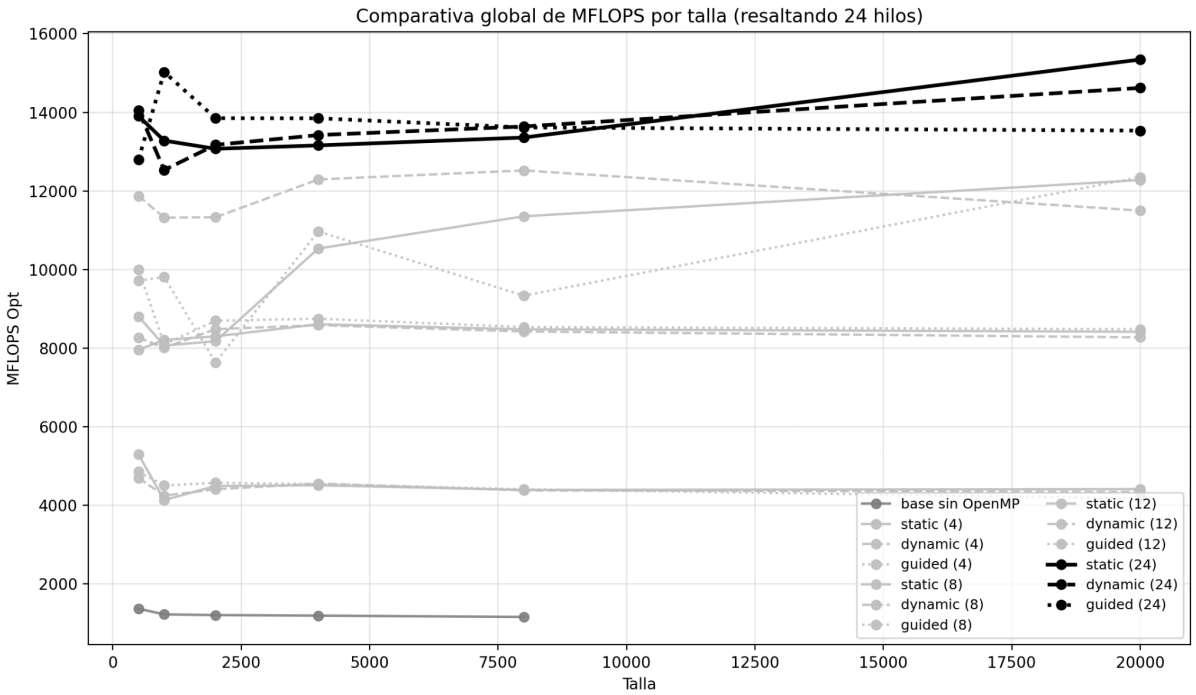
static

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,005684	13905,94697
1000	291.580.012	0,021958	13279,22961
2000	1.117.160.012	0,08547	13071,48872
4000	4.370.320.012	0,33215	13157,70938
8000	17.284.640.012	1,2768	13537,78677
20000	107.291.600.012	6,9927	15343,37238

Finalmente, al ejecutar el algoritmo con 24 hilos, se observa que el rendimiento alcanza su techo máximo (aproximadamente 13.500 MFLOPS para la talla N=8000), lo que se traduce en un SpeedUp cercano a 12x respecto al caso secuencial. Es fundamental destacar que, aunque hemos triplicado el número de hebras respecto al caso de 8 hilos, el rendimiento no se ha triplicado. Este estancamiento en la escalabilidad se justifica por la arquitectura del servidor: la máquina cuenta con un total de 12 núcleos físicos reales. Al lanzar 24 hilos de OpenMP, entra en juego la tecnología Hyper-threading (dos hilos lógicos por cada núcleo físico). Dado que nuestro algoritmo realiza un uso intensivo de las unidades de coma flotante y de la memoria caché, los hilos lógicos que comparten un mismo núcleo físico compiten por estos recursos. Por lo tanto, el rendimiento máximo queda limitado por el hardware físico, impidiendo alcanzar un SpeedUp lineal de 24x.

Por otro lado, al evaluar las políticas de planificación con esta cantidad máxima de hilos, el escenario cambia drásticamente respecto a las pruebas con 4 u 8 hebras. Como se demostró empíricamente al aumentar la talla a N=20.000, el uso de 24 hilos concurrentes saca a relucir el sobrecoste (overhead) de las planificaciones dinámicas. Tener a 24 trabajadores accediendo constantemente a la cola de tareas compartida genera una fuerte contención y tiempos de espera (bloqueos). En este escenario de máximo estrés y con una carga de trabajo matemáticamente regular, la política static se consolida como la opción más eficiente, al eliminar por completo la necesidad de sincronización durante la ejecución de los bucles.





Caso prueba óptimo

Buscando optimizar aún más el rendimiento, decidimos realizar un experimento modificando el código original. Lo que hicimos fue agrupar los bucles principales de talla N en uno solo, con el objetivo de reducir las veces que el sistema tiene que organizar y sincronizar los hilos. Para las tallas pequeñas y medianas, esta nueva versión resultó ser mucho más eficiente; de hecho, en la talla N=8000 logramos mejorar el tiempo del código original en 30 segundos. Sin embargo, al probar con tallas grandes, vimos que la memoria caché L3 de 15 MB del servidor se ve superada por la cantidad de datos. Al saturarse la caché, el procesador pierde tiempo extra yendo a buscar los datos a la memoria principal, lo que hace que en estos casos el código modificado acabe siendo más lento que el original.

talla	FLOPs Totales	Tiempo Opt	MFLOPS Opt.
500	79.040.012	0,005836	13544,0588
1000	291.580.012	0,021527	13544,78829
2000	1.117.160.012	0,07720	14471,07834
4000	4.370.320.012	0,28237	15477,28162
8000	17.284.640.012	1,0957	15774,54301
20000	107.291.600.012	8,0078	13398,33638